

SET-2 — MASTER PRACTICAL GUIDE

E-Ticketing System

Windows PC • Eclipse • Git/GitHub • Maven • Tomcat • Docker

Repository	https://github.com/ssvkotamraju/E-ticketing.git
Coverage	28 individual task entries, each followed immediately by an answer
Exam strategy	Command-first, GUI-supported, with older-version fallbacks and examiner-proof checks

Use this as the exam-day walkthrough: open Git Bash/Command Prompt, Eclipse, Docker Desktop, and a browser side by side. For every task, perform the command, verify the result, and only then move on.

HOW TO USE THIS MASTER PDF

This guide is written for a Windows PC and is designed for practical-exam demonstration. Commands are the most version-tolerant method; GUI routes are included where Eclipse/Docker Desktop can be used. Exact menu labels can differ slightly between old and new releases, so use the command equivalent if a GUI label is missing.

Important source limitation: several exam questions refer to a “provided pom.xml”, but the uploaded question papers contain the questions, not the actual POM file. For those questions, this guide deliberately shows how to derive the exact value from the supplied POM instead of inventing coordinates, WAR names, or context paths.

1. Windows tool setup

Tool	What to verify	Windows command / GUI
Java JDK	Use a JDK, not only a JRE.	<code>java -version</code> <code>javac -version</code>
Maven	Maven available to the terminal, or Maven Wrapper exists.	<code>mvn -version</code> <code>mvnw.cmd -version</code>
Git	Git Bash/Command Prompt can run Git.	<code>git --version</code>
Eclipse	Maven integration (m2e) and Java support.	Window → Preferences → Java
Tomcat	Choose a Tomcat version compatible with the project.	Servers view → New → Apache Tomcat
Docker	Docker daemon/Desktop is running.	<code>docker version</code> Docker Desktop GUI

If `mvn` is not recognized

Either install/configure Maven and add its **bin** directory to Windows PATH, or check whether the repository contains **mvnw.cmd**. The wrapper is often the safest option for older student projects because it uses the project's expected Maven distribution.

```
where mvn
mvn -version
mvnw.cmd -version
```

Windows GUI: Control Panel / Settings → System → Advanced system settings → Environment Variables → create MAVEN_HOME=C:\apache-maven-3.x.x → edit Path → add %MAVEN_HOME%\bin → reopen terminal.

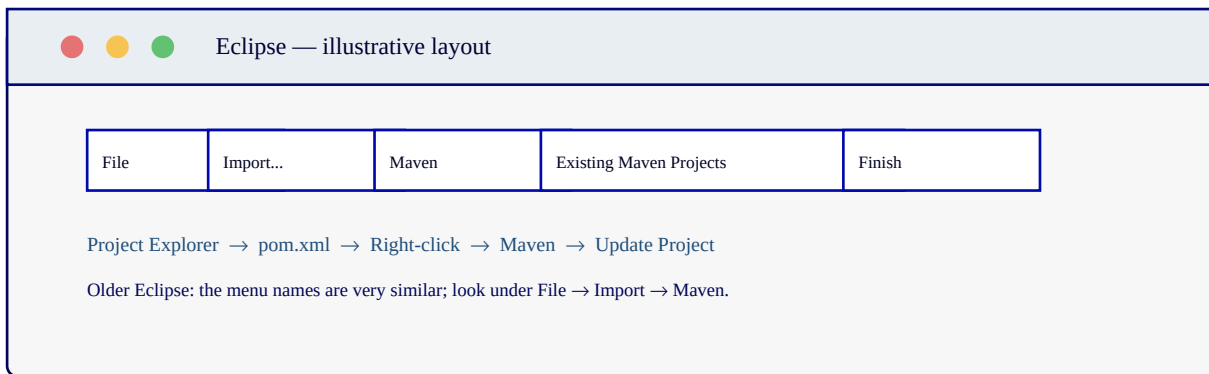
Java compatibility rule

Always run **mvn -version**, not only **java -version**, because Maven reports the Java home it actually uses. If an old project expects Java 8/11 and a newer JDK is installed, align the JDK used by Maven and Eclipse with the project's compiler settings. Do not blindly change the POM until you know the required source/target/release level.

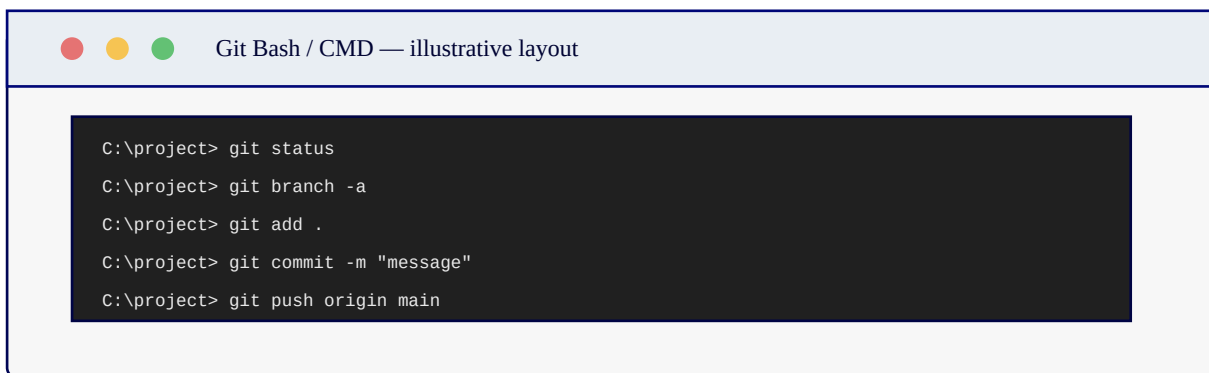
Tomcat compatibility warning

Older JSP/Servlet projects commonly use the **javax.servlet** API. Tomcat 10+ moved to the **jakarta.servlet** namespace. If the project imports `javax.servlet.*`, Tomcat 9 (or another compatible javax-based server) is usually the safer exam choice unless the code has already been migrated to Jakarta.

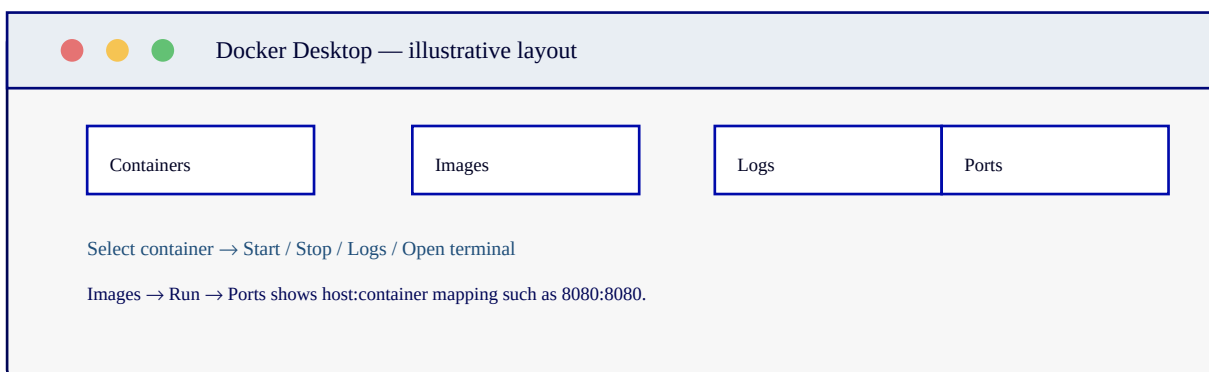
2. GUI routes you can demonstrate to the examiner



Eclipse Maven import: **File** → **Import** → **Maven** → **Existing Maven Projects** → Browse to the cloned folder → select pom.xml → Finish. Then right-click the project → Maven → Update Project. For older Eclipse releases, the wording is almost identical.



Git tasks are best demonstrated in Git Bash because every advanced command in these papers is available there. GitHub Desktop/Eclipse EGit can perform many common operations, but rebase, reflog, patch, and detailed recovery are much easier to prove from Git Bash.



Docker Desktop is useful for showing Images, Containers, Logs, and port mappings. If the lab machine has an older Docker Desktop layout, the same objects may appear under older tabs/menu names. The CLI remains the most reliable fallback.

3. Universal exam command sheet

Area	Commands
Git basics	<code>git clone URL</code> <code>cd project</code> <code>git status</code> <code>git remote -v</code> <code>git branch -a</code>
Branching	<code>git switch -c feature/name</code> # older Git: <code>git checkout -b feature/name</code>
Inspect changes	<code>git diff</code> <code>git diff --cached</code> <code>git status</code>
Commit	<code>git add file</code> <code>git commit -m "message"</code> <code>git log -1 --oneline</code>
Remote sync	<code>git fetch origin</code> <code>git merge origin/main</code> <code>git pull origin main</code>
Rebase	<code>git fetch origin</code> <code>git rebase origin/main</code> <code>git rebase --abort</code>
Undo safely	<code>git revert <hash></code> <code>git reset --soft HEAD~1</code> <code>git reflog</code>
Stash	<code>git stash push -m "work"</code> <code>git stash list</code> <code>git stash pop</code>
Compare/history	<code>git diff main feature -- file</code> <code>git log --oneline --graph --decorate --all</code>
Maven	<code>mvn -version</code> <code>mvn clean package</code> <code>mvn dependency:tree</code> <code>mvn dependency:analyze</code> <code>mvn clean package -X</code>
Tests	<code>mvn test</code> <code>mvn -Dtest=ClassName test</code> <code>mvn clean package -DskipTests</code>
Docker	<code>docker build -t app:v1 .</code> <code>docker run -d --name app -p 8080:8080 app:v1</code> <code>docker ps</code> <code>docker logs app</code>
Docker Hub	<code>docker login</code> <code>docker tag app:v1 user/app:v1</code> <code>docker push user/app:v1</code> <code>docker logout</code>

4. Universal troubleshooting decision tree

Maven failure: read the first meaningful [ERROR] → identify the phase/goal → inspect the POM coordinates/configuration → run `dependency:tree/effective-pom` → check JDK with `mvn -version` → clean → rebuild with `-X`.

Browser failure: prove Maven created the artifact → prove the server started → prove the port is listening → prove the context path → read server logs → then inspect JSP/application code.

Docker failure: prove Docker daemon → prove Dockerfile/context → prove image exists → prove container exists → inspect `docker ps -a` → inspect docker logs → inspect port mapping.

5. COMPLETE TASK-BY-TASK QUESTION → ANSWER GUIDE

The task wording below is kept close to the uploaded question paper while being shortened where a long scenario is repeated. Every numbered/sub-numbered practical requirement is represented. Where the source asks for an exact value that depends on an unseen POM, the answer explicitly tells you how to derive it.

Q0 — TASK / QUESTION

E-Ticketing scenario: identify the core booking-system requirements and the operational goal.

ANSWER / PROCEDURE

Translate the scenario into functional and non-functional requirements, actors, core use cases, and deployment needs. For a web hospital/ticketing/recruitment system, include authentication, role-based access, CRUD workflows, reports, security, usability, and concurrent-user performance.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

1(a) — TASK / QUESTION

Clone the E-Ticketing repository and import it into Eclipse.

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/ssvкотamraju/E-ticketing.git
cd E-ticketing
git status
git remote -v
```

GUI route: Eclipse → File → Import → Maven → Existing Maven Projects; for an already imported project use Right-click → Maven → Update Project.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

1(b) — TASK / QUESTION

Identify and resolve Maven dependency-related errors after import.

ANSWER / PROCEDURE

In Eclipse, right-click the project → Maven → Update Project... (older Eclipse/m2e versions use the same Maven menu). Check Force Update of Snapshots/Releases when a stale dependency is suspected, then Project → Clean.

GUI route: Eclipse → File → Import → Maven → Existing Maven Projects; for an already imported project use Right-click → Maven → Update Project.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(c) — TASK / QUESTION

Fix a Java compiler configuration in pom.xml that does not match the installed Java version.

ANSWER / PROCEDURE

Check `<maven.compiler.source>` / `<maven.compiler.target>` or the `maven-compiler-plugin` configuration. Use a compiler level supported by the installed JDK and by the codebase. In older projects you will often see `1.8`; modern Maven Compiler Plugin versions also support a `release` setting.

Commands / configuration

```
mvn -version
mvn help:effective-pom
mvn clean compile
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(d) — TASK / QUESTION

Use Maven error output to identify the root cause of a compilation failure.

ANSWER / PROCEDURE

Run Maven with debug logging and preserve the first meaningful error. Do not start by changing random POM values.

Check the first `[ERROR]`, the goal/phase named, and the `Caused by:` chain.

Commands / configuration

```
mvn clean package -X
mvn clean package -e
mvn -version
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(e) — TASK / QUESTION

The application must be a web application but Maven creates the wrong package type. What do you inspect/change?

ANSWER / PROCEDURE

Inspect the top-level ``<packaging>`` element. For a traditional JSP/Servlet web application it is normally ``war``; if omitted, Maven defaults to ``jar``. Changing the packaging changes the lifecycle and generated artifact.

Commands / configuration

```
mvn help:evaluate -Dexpression=project.packaging -q -DforceStdout
mvn clean package
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(f) — TASK / QUESTION

After changing pom.xml, explain why a clean Maven build helps and which command to use.

ANSWER / PROCEDURE

A clean build removes the old ``target`` directory, so stale classes, old generated files, and previous artifacts cannot hide a POM/configuration change.

Commands / configuration

```
mvn clean
mvn clean package
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(g) — TASK / QUESTION

Maven says BUILD SUCCESS but the E-Ticketing application does not open correctly. What checks do you perform?

ANSWER / PROCEDURE

A successful Maven build only proves the build lifecycle completed. Verify the WAR/JAR exists, start the server, confirm the server log says it started, confirm the expected port is listening, then open the correct context path. Check browser/server logs for 404/500 errors.

Commands / configuration

```
dir target
netstat -ano | findstr :8080
tasklist | findstr /I java
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(h) — TASK / QUESTION

Create a production package without running tests.

ANSWER / PROCEDURE

For a temporary package when tests cannot run, use `-DskipTests`. It compiles tests but skips running them. `-Dmaven.test.skip=true` skips compiling and running tests and is more aggressive. Explain the risk: the artifact can be produced without current test evidence.

Commands / configuration

```
mvn clean package -DskipTests
mvn clean package -Dmaven.test.skip=true
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

1(i) — TASK / QUESTION

A dependency cannot be downloaded. Determine whether the cause is the declaration, repository, or network.

ANSWER / PROCEDURE

Run Maven with debug logging and preserve the first meaningful error. Do not start by changing random POM values. Check the first `[ERROR]`, the goal/phase named, and the `Caused by:` chain.

Commands / configuration

```
mvn clean package -X
mvn clean package -e
mvn -version
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(j) — TASK / QUESTION

Run the project's tests after fixing ticket-booking logic.

ANSWER / PROCEDURE

Run the normal test phase. For one test class use Surefire's `-Dtest=ClassName` selector.

Commands / configuration

```
mvn test
mvn -Dtest=TestClassName test
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

2(a) — TASK / QUESTION

Create the first Git commit for a local railway ticket-booking folder.

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/ssvkotamraju/E-ticketing.git
cd E-ticketing
git status
git remote -v
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(b) — TASK / QUESTION

Create a separate branch for train-search without modifying main.

ANSWER / PROCEDURE

Create and switch in one operation. This is the modern command; on older Git use the checkout equivalent.

Commands / configuration

```
git switch -c feature/train-search
git checkout -b feature/train-search
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(c) — TASK / QUESTION

Review modified ticket-booking page and new CSS before committing.

ANSWER / PROCEDURE

Use `git status` first. It separates modified-but-unstaged files, staged files, and untracked files. For exact line-level edits use `git diff`; for staged edits use `git diff --cached`.

Commands / configuration

```
git status
git diff
git diff --cached
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(d) — TASK / QUESTION

Discard uncommitted changes to the train-search page.

ANSWER / PROCEDURE

For an uncommitted file, restore the working-tree copy from HEAD. This permanently discards the uncommitted edits, so show the examiner `git diff` first if you want proof of what is being discarded.

Commands / configuration

```
git diff -- "train-search page"
git restore -- "train-search page"
git checkout -- "train-search page"
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(e) — TASK / QUESTION

Move an unpushed branch back to an earlier experimental commit.

ANSWER / PROCEDURE

For unpushed experimental commits, move the branch pointer back to the chosen commit. `--hard` also resets tracked working files, so verify the hash before running it.

Commands / configuration

```
git log --oneline
git reset --hard <earlier-commit>
git status
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(f) — TASK / QUESTION

Identify and delete unused local/remote feature branches.

ANSWER / PROCEDURE

List merged branches, then delete local branches that are no longer needed. Use ``-d`` for a safety check; use ``-D`` only when you intentionally want to force deletion. Remote deletion is a separate command.

Commands / configuration

```
git branch --merged main
git branch -d <old-branch>
git push origin --delete <old-branch>
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run ``git status`` and/or ``git log -1 --oneline``; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(g) — TASK / QUESTION

Merge train-search and passenger-registration branches into main.

ANSWER / PROCEDURE

Switch to main, make sure it is current, merge the feature branch, resolve conflicts if Git stops, then commit the merge if required.

Commands / configuration

```
git switch main
git pull origin main
git merge feature/train-search
git status
git push origin main
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run ``git status`` and/or ``git log -1 --oneline``; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(h) — TASK / QUESTION

Create, inspect, and apply a Git patch from a teammate.

ANSWER / PROCEDURE

For a patch file, first validate it without changing files. Then apply it. If the patch contains Git commits produced by ``format-patch``, use ``git am`` instead of ``git apply``.

Commands / configuration

```
git apply --check teammate.patch
git apply teammate.patch
git diff
git add .
git commit -m "Apply teammate patch"
git am teammate.patch
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run ``git status`` and/or ``git log -1 --oneline``; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(i) — TASK / QUESTION

Resolve a patch that cannot be applied cleanly because local changes conflict.

ANSWER / PROCEDURE

Check whether the patch applies cleanly, inspect the local version, update from the correct base if needed, and apply with 3-way fallback when appropriate. Resolve conflict markers, test, stage, and commit.

Commands / configuration

```
git apply --check teammate.patch
git fetch origin
```

```
git diff
git apply --3way teammate.patch
git status
git diff
git add .
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(j) — TASK / QUESTION

Configure the remote repository and push the railway project/history to GitHub.

ANSWER / PROCEDURE

Push the branch and then compare the local commit ID with the remote-tracking branch after fetching. Equal hashes mean both point to the same commit.

Commands / configuration

```
git push -u origin main
git fetch origin
git rev-parse HEAD
git rev-parse origin/main
git status
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

3(i) — TASK / QUESTION

Create a Dockerfile and build an image named OTS for Docker Hub.

ANSWER / PROCEDURE

For a JSP/Servlet WAR, use Tomcat as the runtime. The WAR is copied into Tomcat's `webapps` directory. A multi-stage build is cleaner because Maven is used only in the build stage.

Commands / configuration

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM tomcat:9-jdk17
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY --from=build /app/target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(ii) — TASK / QUESTION

Run the OTS image and expose port 8080; verify localhost:8080.

ANSWER / PROCEDURE

Start detached, map host 8080 to container 8080, and name the container so it is easy to demonstrate.

Commands / configuration

```
docker run -d --name ots-container -p 8080:8080 OTS:v1
docker ps
docker port ots-container
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

3(a) — TASK / QUESTION

docker build fails while creating OTS. Explain the troubleshooting procedure.

ANSWER / PROCEDURE

Read the build error from the first failing instruction, check the build context and file names, confirm the Dockerfile is in the current directory, and rebuild without guessing. For runtime failures, inspect `docker ps -a` and `docker logs`.

Commands / configuration

```
docker build -t app:v1 .
docker ps -a
docker logs <container>
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(b) — TASK / QUESTION

The E-Ticketing app listens on container port 8080 but localhost:8080 fails. Explain port mapping.

ANSWER / PROCEDURE

A container port is not automatically published to the Windows host. `-p 8080:8080` means HOST_PORT:CONTAINER_PORT. Verify the mapping with `docker port` and then browse to localhost:8080.

Commands / configuration

```
docker run -d --name app -p 8080:8080 app:v1
docker port app
netstat -ano | findstr :8080
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(c) — TASK / QUESTION

The OTS container stops immediately. Which commands/logs should be inspected?

ANSWER / PROCEDURE

Use logs for the application/server's stdout/stderr and inspect the container state. If it exits immediately, `docker ps -a` plus `docker logs` is the fastest diagnosis.

Commands / configuration

```
docker ps -a
docker logs ots-container
docker inspect ots-container
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(d) — TASK / QUESTION

Explain how Docker removes Java/server configuration differences between developers.

ANSWER / PROCEDURE

Before writing the Dockerfile, determine whether the project produces a JAR or WAR, the Java version, the server/runtime (Tomcat for JSP/Servlet apps), the artifact name under `target`, the application port, and the startup command. Then choose the matching base image and copy the artifact into the correct runtime location.

Commands / configuration

```
mvn package
dir target
docker version
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(e) — TASK / QUESTION

Deploy a Maven-generated WAR into a Tomcat-based Docker container on port 8080.

ANSWER / PROCEDURE

For a JSP/Servlet WAR, use Tomcat as the runtime. The WAR is copied into Tomcat's `webapps` directory. A multi-stage build is cleaner because Maven is used only in the build stage.

Commands / configuration

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM tomcat:9-jdk17
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY --from=build /app/target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

6. EXAMINER DEMONSTRATION CHECKLIST

- Open the project folder and prove the repository/remote is correct.
- Open pom.xml and point to groupId, artifactId, version, packaging, dependencies and plugins when asked.
- Run mvn -version and, if required, java -version/javac -version.
- Build with mvn clean package (or the exact lifecycle command in the question).
- Show target/ and the generated WAR/JAR.
- In Eclipse, use Maven → Update Project when dependencies are stale.
- For Git, show status/diff before commit and log after commit.
- For rebase/merge questions, show the branch name and final clean status.
- For Docker, show docker images, docker ps -a, docker logs and port mapping as appropriate.
- Open http://localhost:8080 only after proving the server/container is running.
- For Docker Hub, show tag → login → push → repository verification.

7. OLD-VERSION FALLBACKS

If you see...	Use this fallback
Old Git without `git switch`	Use `git checkout -b branch` / `git checkout branch`.
Old Eclipse	Look under File → Import → Maven and Window → Show View → Servers; labels vary slightly.
Old Maven project	Check for `mvnw.cmd`; otherwise use a Maven version compatible with the project/JDK.
Old Tomcat project using javax.servlet	Prefer a javax-compatible Tomcat release (commonly Tomcat 9) unless the project is migrated to Jakarta.
Docker Desktop GUI differs	Use the CLI; the core objects remain images, containers, logs, and port mappings.
PowerShell command differs	Git Bash is the simplest universal shell for the Git commands in these papers.

Official references used for the practical workflow

Git for Windows: <https://git-scm.com/install/windows>

Eclipse documentation: <https://www.eclipse.org/documentation/>

Maven Compiler Plugin: <https://maven.apache.org/plugins/maven-compiler-plugin/>

Maven Dependency Plugin: <https://maven.apache.org/plugins/maven-dependency-plugin/>

Docker Desktop for Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>